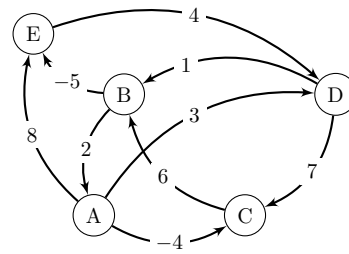


Week 12. Problem set (solutions by Karim Khabibrakhmanov)

1. Run the Floyd-Warshall algorithm [CLRS, §23.2] on the following graph.

Consider vertices alphabetically (A, B, C, D, E). Show the state of distance matrix D after each iteration of outer loop in the algorithm. Since the graph has 5 vertices, you must provide five 5×5 matrices in your answer. No justification is required.



Answer:

0	∞	-4	3	8
2	0	∞	∞	-5
∞	6	0	∞	∞
∞	1	7	0	∞
∞	∞	∞	4	0

$$\mathbf{D}^{(1)} =$$

0	∞	-4	3	8
2	0	-2	5	-5
∞	6	0	∞	∞
∞	1	7	0	∞
∞	∞	∞	4	0

$$\mathbf{D}^{(2)} =$$

0	∞	-4	3	8
2	0	-2	5	-5
8	6	0	11	1
3	1	-1	0	-4
∞	∞	∞	4	0

$$\mathbf{D}^{(3)} =$$

0	2	-4	3	-3
2	0	-2	5	-5
8	6	0	11	1
3	1	-1	0	-4
∞	∞	∞	4	0

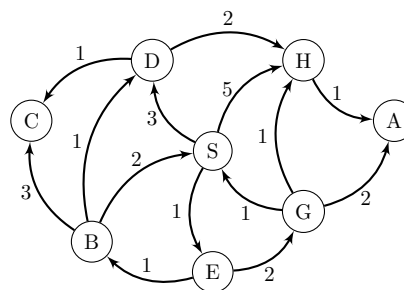
$$\mathbf{D}^{(4)} =$$

0	2	-4	3	-3
2	0	-2	5	-5
8	6	0	11	1
3	1	-1	0	-4
7	5	3	4	0

$$\mathbf{D}^{(5)} =$$

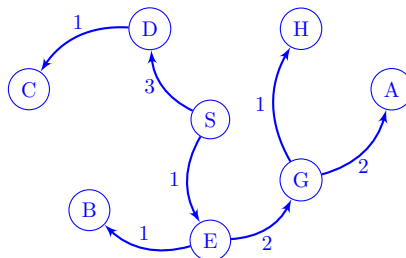
0	2	-4	1	-3
2	0	-2	-1	-5
8	6	0	5	1
3	1	-1	0	-4
7	5	3	4	0

2. Run Dijkstra's algorithm [CLRS, §22.3] on the following graph $\mathcal{G}$, starting at vertex S , and answer the questions below.



- (a) Draw a shortest paths tree of the graph $\mathcal{G}$.

Answer:



- (b) Is the shortest paths tree for graph $\mathcal{G}$ unique?

Answer: No

- (c) Assuming that the algorithm is using Binary Heap implementation [CLRS, §6] of a priority queue, show the state of the priority queue after each iteration of the algorithm (i.e. after adding each new vertex to the shortest paths tree). The graph contains 8 vertices, which means that your solution must provide 8 states of the Binary heap. Each heap state must be represented as an array.

For example, a binary min-heap containing key-value pairs $\langle 3, a \rangle, \langle 2, b \rangle, \langle 1, c \rangle, \langle 4, d \rangle$ may be represented as follows:

1, c	3, a	2, b	4, d
------	------	------	------

Answer:

1. Add S	1, <i>E</i>	3, <i>D</i>	5, <i>H</i>	
2. Add E	2, <i>B</i>	3, <i>G</i>	3, <i>D</i>	5, <i>H</i>
3. Add B	3, <i>G</i>	5, <i>H</i>	3, <i>D</i>	5, <i>C</i>
4. Add G	3, <i>D</i>	4, <i>H</i>	5, <i>C</i>	5, <i>A</i>
5. Add D	4, <i>H</i>	5, <i>A</i>	4, <i>C</i>	
6. Add H	4, <i>C</i>	5, <i>A</i>		
7. Add C	5, <i>A</i>			
8. Add A				

3. Consider a graph $\mathcal{G} = (V, E)$ that represents a computer network. Each edge (u, v) represents a physical cable connecting two computers and has a *length* $\ell(u, v)$ and a *bandwidth* $b(u, v)$.

The *length of a path* P is computed as the sum of lengths of all edges in P . The *bandwidth of a path* P is computed as the minimum of bandwidth values for all edges in P .

Given two paths P_1, P_2 in the $\mathcal{G}$ with lengths ℓ_1, ℓ_2 and bandwidth values b_1, b_2 , we consider P_1 to be *better* than P_2 when $\ell_1 < \ell_2$ or ($\ell_1 = \ell_2$ and $b_1 > b_2$).

Consider a modification of Dijkstra's algorithm [CLRS, §22.3] that finds the best paths in such a computer network from a single source vertex. Answer the following questions:

- (a) What should serve as key (priority) in the priority queue in the modified Dijkstra's algorithm?

Answer: In the **Priority Queue**, the $\text{length}(\ell)$ should be given priority, followed by $\text{bandwidth}(b)$. The $\text{bandwidth}(b)$ takes priority when the $\text{lengths}(\ell)$ of two paths are equal. Key priority is $\text{lengths}(\ell)$.

- (b) Is it still possible to rely on DECREASE-KEY in the implementation? If yes, explain how. If no, explain why. In either case provide a brief justification (1–2 sentences).

Answer: Yes

Justification:

The **Decrease-Key** function preserves strict order and monotony for a pair (length, bandwidth):

- i. The shorter length takes precedence
- ii. With the same length, priority goes to higher bandwidth.

- (c) Correctness of Dijkstra's algorithm [CLRS, Theorem 22.6] has a constraint on the weight function w . What constraint on functions ℓ and b is necessary for the correctness of the modified Dijkstra's algorithm?

Answer:

For the algorithm to work correctly, we need:

- i. All edge lengths $\ell(u, v)$ to be non-negative ($\ell(u, v) \geq 0$)
- ii. All bandwidths $b(u, v)$ to be monotonic (non-increasing along any path)

References

- [CLRS] Cormen, T.H., Leiserson, C.E., Rivest, R.L. and Stein, C., 2022. *Introduction to algorithms, Fourth Edition*. MIT press.
- [GTG] M. T. Goodrich, R. Tamassia, and M. H. Goldwasser. *Data Structures and Algorithms in Java*. WILEY 2014.